

Java OOP — Inheritance Onwards

Inheritance, constructors with inheritance, super, method overriding, and runtime polymorphism.

1. What is Inheritance?

Inheritance allows one class to acquire properties and methods of another class. It promotes code reuse.

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

2. Why Do We Need Inheritance?

Inheritance avoids duplicate code by putting common properties and methods in a parent class.

```
class Animal {
    String name;

    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
}

class Cat extends Animal {
}
```

3. extends Keyword

Java uses **extends** to establish inheritance. Parent = superclass/base class; child = subclass/derived class.

```
class Parent {
}

class Child extends Parent {
}
```

4. Simple Inheritance Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {

```

```

        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}

```

5. Inheritance with Variables

```

class Animal {
    String name;
    int age;
}

class Dog extends Animal {
    String breed;
}

Dog d = new Dog();
d.name = "Tommy";
d.age = 5;
d.breed = "Labrador";

```

6. IS-A Relationship

Inheritance generally represents an IS-A relationship.

```

Dog IS-A Animal
Student IS-A Person
Manager IS-A Employee

```

```

IS-A → Inheritance
HAS-A → Composition

```

7. Types of Inheritance

Java class inheritance commonly appears as single, multilevel, and hierarchical. Multiple inheritance through classes is not supported.

Single: $A \rightarrow B$

Multilevel: $A \rightarrow B \rightarrow C$

Hierarchical:

```

    A
   / \
  B   C

```

Multiple inheritance through classes:
not supported in Java.

8. Inheritance and private Members

A private member cannot be directly accessed from a child class. Access modifiers therefore matter when using inheritance.

```

class Person {
    private String name;
}

class Student extends Person {
    // name cannot be directly accessed here
}

```

9. Inheritance with Constructors

Constructors are not inherited. When a child object is created, the parent constructor executes before the child constructor.

```
class Person {
    Person() {
        System.out.println("Person constructor");
    }
}

class Student extends Person {
    Student() {
        System.out.println("Student constructor");
    }
}

Student s = new Student();
```

Output:
Person constructor
Student constructor

10. Constructor Execution Order

For multilevel inheritance, constructors execute from the topmost parent down to the current child.

```
class A {
    A() { System.out.println("A"); }
}

class B extends A {
    B() { System.out.println("B"); }
}

class C extends B {
    C() { System.out.println("C"); }
}

C obj = new C();
```

Output:
A
B
C

11. super() Keyword

super() calls the parent class's no-argument constructor. If an accessible no-argument parent constructor exists and super() is omitted, Java inserts it automatically as the first statement.

```
class Person {
    Person() {
        System.out.println("Person constructor");
    }
}

class Student extends Person {
    Student() {
        super();
        System.out.println("Student constructor");
    }
}
```

12. Parent Constructor with Parameters

If the parent has only a parameterized constructor, the child must explicitly call it using super(arguments).

```

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    int marks;

    Student(String name, int marks) {
        super(name);
        this.marks = marks;
    }
}

```

13. Complete Inheritance Constructor Example

```

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Person constructor");
    }

    void displayPerson() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Student extends Person {
    int marks;

    Student(String name, int age, int marks) {
        super(name, age);
        this.marks = marks;
        System.out.println("Student constructor");
    }

    void displayStudent() {
        System.out.println("Marks: " + marks);
    }
}

public class InheritanceConstructor {
    public static void main(String[] args) {
        Student s = new Student("Sanjay", 21, 85);
        s.displayPerson();
        s.displayStudent();
    }
}

```

14. this vs super

this refers to the current object/current class context. super refers to the parent class.

```

this.name = name;
→ current object's field

```

```

super(name, age);
→ parent constructor

```

```

this → current

```

super → parent

15. super Can Call Parent Methods

super.method() specifically calls the parent implementation of a method.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog sound");
    }

    void show() {
        super.sound();
    }
}
```

16. super with Variables

If parent and child have fields with the same name, super.field refers to the parent's field.

```
class Person {
    int age = 50;
}

class Student extends Person {
    int age = 20;

    void display() {
        System.out.println(age);
        System.out.println(super.age);
    }
}
```

Output:

20
50

17. Method Overriding

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

18. Rules of Method Overriding

Inheritance must exist. The method name and parameter list must match. The child provides a new implementation.

```
Parent:
void sound()
```

```
Child:
@Override
void sound()
```

Same signature + inheritance
→ overriding

19. Overloading vs Overriding

Overloading uses the same method name with different parameter lists. Overriding uses inheritance and the same method signature.

```
Overloading:
add(int, int)
add(int, int, int)
```

```
Overriding:
Parent → sound()
Child → sound()
```

20. @Override Annotation

@Override tells Java that you intend to override a parent method and helps detect signature mistakes.

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

21. Parent Reference + Child Object

A parent reference can refer to a child object. For an overridden instance method, the child's implementation executes at runtime.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
```

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
Animal a = new Dog();
a.sound();
```

Output:
Dog barks

22. Runtime Polymorphism

Runtime polymorphism occurs when the overridden method that executes is determined by the actual object at runtime.

Parent reference
+

Child object
+
Overridden method
↓
Child method executes

```
Animal a = new Dog();  
a.sound();
```

→ Dog.sound()

23. Important Interview Points

Constructors are not inherited. Parent constructors execute before child constructors. `super()` calls a parent constructor and must be first in a constructor. Private and final methods cannot be overridden. Static methods are hidden rather than dynamically overridden.

24. Practice Problem

Create Vehicle with `start()`. Create Car extends Vehicle and Bike extends Vehicle. Override `start()` in both. In `main()`, use Vehicle references with Car and Bike objects and call `start()`. Expected: Car starts with key; Bike starts with self-start.

25. Key Takeaways

Inheritance enables code reuse and represents IS-A. `extends` creates class inheritance. Parent constructors execute before child constructors. `super()` calls the parent constructor. `super.method()` calls a parent method. Method overriding changes inherited behavior. Parent reference + child object is the foundation of runtime polymorphism.